

PADRÃO DE PROJETO ESTRUTURAL (GoF)

FLYWEIGHT

Compartilhando objetos com eficiência

FABÍOLA C. POTRICH

Engenharia de Software II • IF/RS Farroupilha

O Que Vamos Ver

1

O problema

Milhões de objetos, pouca memória

2

A ideia central

Como o Flyweight resolve isso

3

Estrutura e funcionamento

UML, código e passo a passo

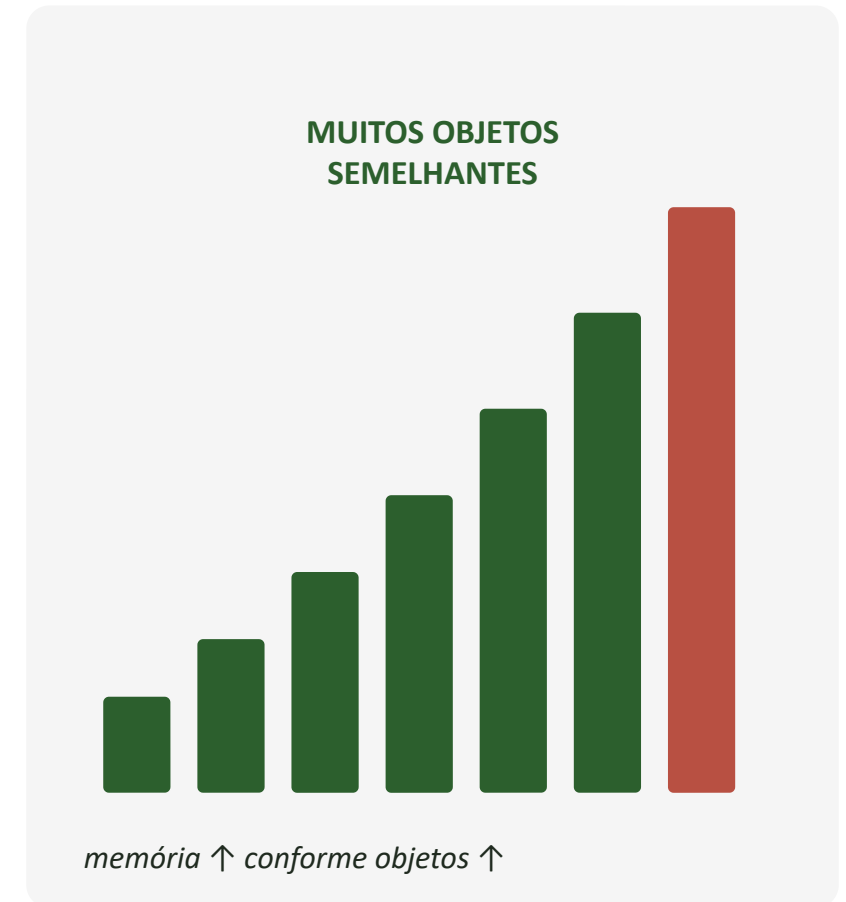
4

Desempenho e uso prático

Números reais e quando aplicar

Quando Muitos Objetos Viram um Problema

- Jogo com 1 milhão de árvores: cada uma guarda malha, textura e cor
- Editor de texto: um objeto novo para cada caractere digitado
- Resultado: o consumo de memória explode rapidamente



Separar o Que se Repete do Que Muda

ESTADO INTRÍNSECO

Compartilhado

- Igual em muitos objetos
- Imutável
- Fica dentro do Flyweight

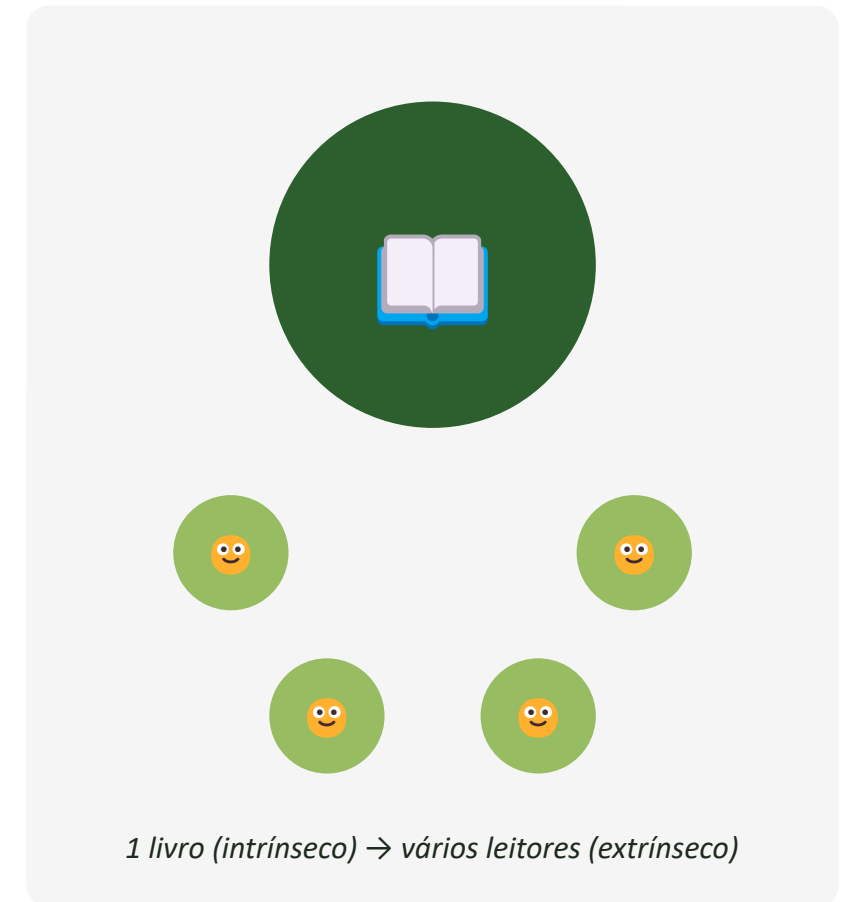
ESTADO EXTRÍNSECO

Passado por fora

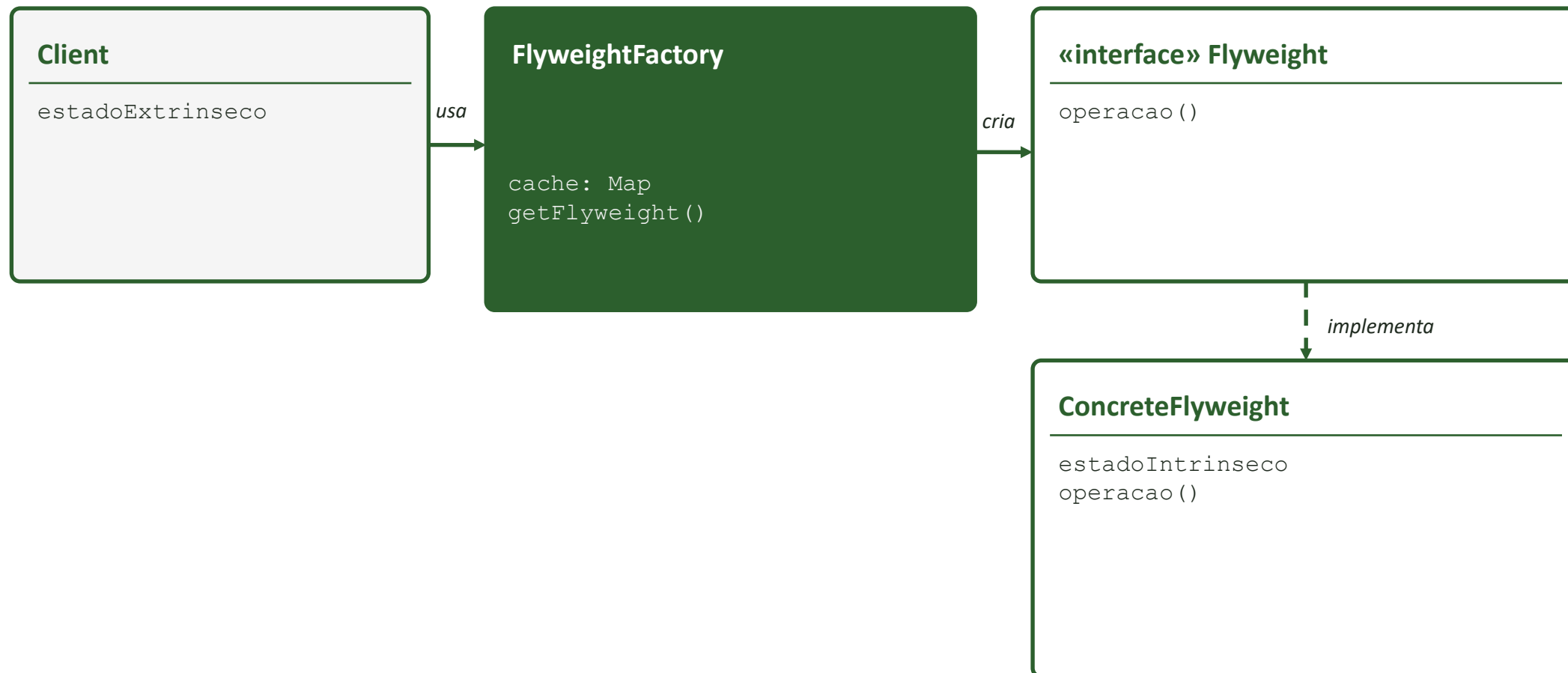
- Varia por objeto
- Guardado pelo cliente
- Passado como parâmetro

Uma Analogia Simples

- Biblioteca: um único livro físico, muitos leitores diferentes
- O conteúdo do livro é compartilhado por todos
- Cada leitor guarda apenas a sua própria página marcada

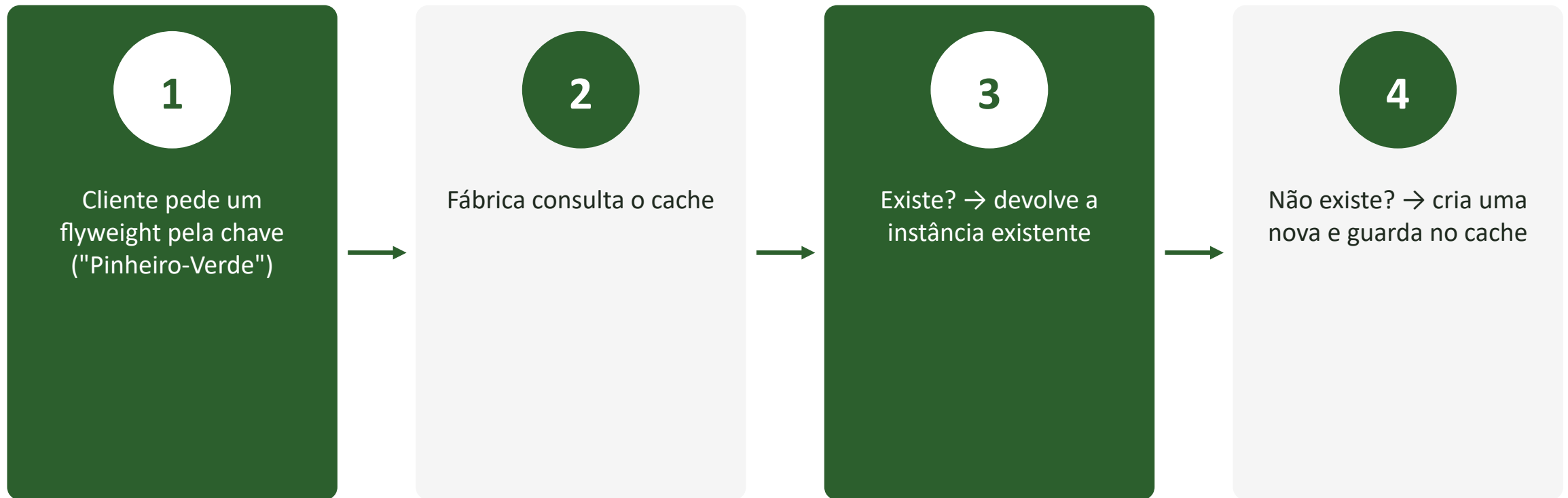


Como as Peças se Encaixam



A fábrica devolve a mesma instância quando a chave já existe no cache.

O Papel da Fábrica

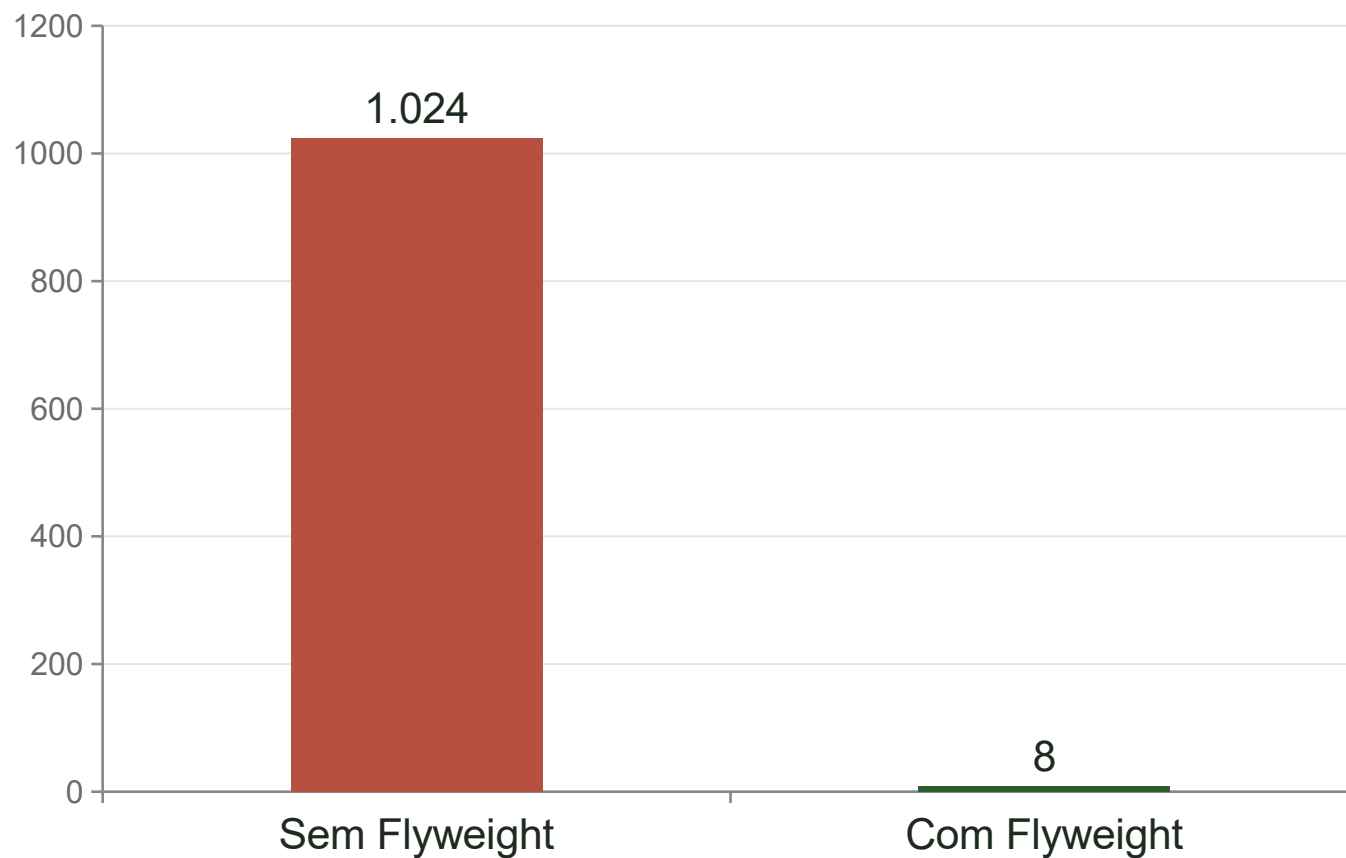


Flyweight em Java

```
class TreeFactory {  
    static Map<String, TreeType> cache = new HashMap<>();  
  
    static TreeType getTreeType(String chave) {  
        return cache.computeIfAbsent(chave,  
            k -> new TreeType(chave));    // cria so se nao existir  
    }  
}
```

- Map guarda o cache de tipos já criados
- computeIfAbsent evita recriar o que já existe
- Tree guarda só (x, y) + a referência ao tipo

O Impacto Real na Memória



1.000.000

árvores na cena

apenas 2

objetos TreeType criados

≈ 125× menos memória

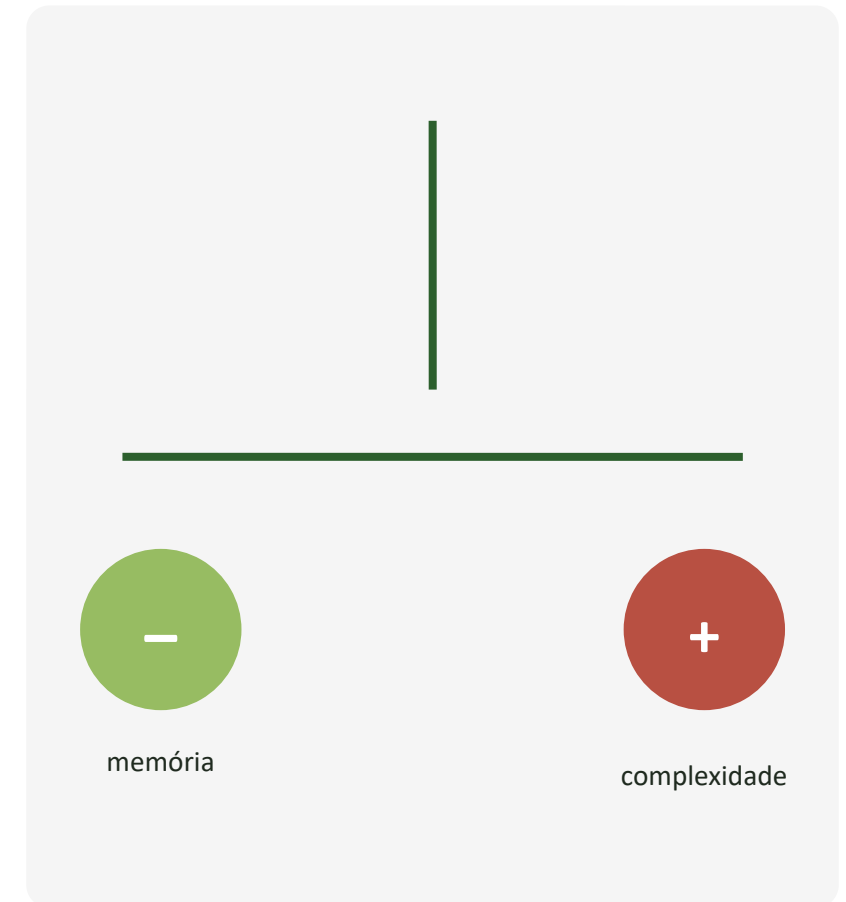
Use Flyweight Quando...

- Existem muitos objetos semelhantes na aplicação
- O custo de memória é um problema real
- Boa parte do estado pode virar extrínseco
- A identidade de cada objeto não importa



O Preço do Compartilhamento

- Código fica mais complexo de organizar
- Pode exigir mais CPU a cada chamada
- Flyweights precisam ser imutáveis
- Depurar fica mais difícil (objetos compartilhados)



Você Já Usa Flyweight

`Integer.valueOf()`

Cacheia valores entre -128 e 127

`String pool`

Literais iguais compartilham a mesma instância

`Jogos e editores gráficos`

Sprites, fontes e partículas reaproveitados

Para Lembrar

- Flyweight compartilha o que se repete e separa o que varia
- Grande ganho de memória, com um custo de complexidade
- Ideal para sistemas com muitos objetos semelhantes

Referências

- GAMMA, HELM, JOHNSON, VLISSIDES. Design Patterns (GoF) — capítulo Flyweight.
 - Refactoring Guru — refactoring.guru/pt-br/design-patterns/flyweight
 - SourceMaking — sourcemaking.com/design_patterns/flyweight
-

Obrigada!